

```

import numpy as np
import geopandas as gpd
import folium
import warnings
import geemap
from shapely.prepared import prep
from shapely.geometry import Point, Polygon
from shapely.ops import unary_union
from scipy import spatial
import ee

<IPython.core.display.HTML object>

ee.Authenticate(auth_mode="localhost")

<IPython.core.display.HTML object>

True

ee.Initialize(project='qgis-proj')

<IPython.core.display.HTML object>

# Initialize Map

Map = geemap.Map()

<IPython.core.display.HTML object>

# Load Dhaka asset

dhaka_fc = (
    ee.FeatureCollection('projects/qgis-proj/assets/bangladesh_adm2')
    .filter(ee.Filter.eq('adm2_name', 'Dhaka'))
)
dhaka=dhaka_fc.geometry()

#Add Layer of Dhaka

Map.addLayer(dhaka_fc, {'color': 'FF0000'}, 'Dhaka District')
Map.centerObject(dhaka, 11)

<IPython.core.display.HTML object>

# Decide on timeline
START='2023-05-01'
END='2025-08-01'

<IPython.core.display.HTML object>

# Landsat Loading

l9 = (ee.ImageCollection('LANDSAT/LC09/C02/T1_L2'))

```

```

        .filterBounds(dhaka)
        .filterDate(START, END)
        .filter(ee.Filter.lt('CLOUD_COVER', 20)))

l8 = (ee.ImageCollection('LANDSAT/LC08/C02/T1_L2')
      .filterBounds(dhaka)
      .filterDate(START, END)
      .filter(ee.Filter.lt('CLOUD_COVER', 80))) #chose 80 here cuz at
20 very few results

# The filter: PROCESSING_LEVEL == 'L2SP' is to ensure that only images
with
# corrected reflectance bands and thermal surface temperature products
are given
# without it, indices calc was off especially LST

collection = l8.merge(l9).filter(ee.Filter.eq('PROCESSING_LEVEL',
'L2SP'))

print("Total Landsat scenes:", collection.size().getInfo())

<IPython.core.display.HTML object>

Total Landsat scenes: 115

# Processing the images

#Taken directly from Google Earth Engine Documentation:
https://developers.google.com/earth-engine/datasets/catalog/LANDSAT_LC
09_C02_T1_L2#colab-python

# Applies scaling factors.
def apply_scale_factors(image):
    optical_bands = image.select('SR_B.').multiply(0.0000275).add(-0.2)
    thermal_bands =
image.select('ST_B.*').multiply(0.00341802).add(149.0)
    return image.addBands(optical_bands, None, True).addBands(
        thermal_bands, None, True
    )

scaled = collection.map(apply_scale_factors)

def maskClouds(image):
    """
    Bit 0: Fill
    Bit 1: Dilated Cloud
    Bit 2: Cirrus
    Bit 3: Cloud
    Bit 4: Cloud Shadow
    """

```

```

qa = image.select('QA_PIXEL')

mask = (
    qa.bitwiseAnd(1 << 1).eq(0)
    .And(qa.bitwiseAnd(1 << 2).eq(0))
    .And(qa.bitwiseAnd(1 << 3).eq(0))
    .And(qa.bitwiseAnd(1 << 4).eq(0))
)

return image.updateMask(mask)

masked = scaled.map(maskClouds)

def add_indices(image):
    ndvi = image.normalizedDifference(['SR_B5',
    'SR_B4']).rename('NDVI')
    ndbi = image.normalizedDifference(['SR_B6',
    'SR_B5']).rename('NDBI')
    #LST in kelvin, converting to celsius
    #ST_B10 → scale → Kelvin → Celsius
    #REF:
    https://www.researchgate.net/post/How\_do\_i\_calculate\_Land\_Surface\_Temperature\_Landsat\_8\_level\_2\_image
    lst = image.select('ST_B10').subtract(273.15).rename('LST')
    return image.addBands([ndvi, ndbi, lst])

processed=scaled.map(add_indices)

<IPython.core.display.HTML object>

# Visualize if clouds were correctly removed or not

before = scaled.median()
after = masked.median()

vis = {
    'bands': ['SR_B4', 'SR_B3', 'SR_B2'],
    'min': 0,
    'max': 0.3
}

Map.addLayer(before.clip(dhaka), vis, 'Before')
Map.addLayer(after.clip(dhaka), vis, 'After')

Map

<IPython.core.display.HTML object>

{"model_id": "4ee8608e835f4d4f8444d1eb89b2d0a0", "version_major": 2, "version_minor": 0}

```

```

# Calculate Indices: NDVI, NDBI, LST

def add_indices(image):
    ndvi = image.normalizedDifference(['SR_B5',
    'SR_B4']).rename('NDVI')
    ndbi = image.normalizedDifference(['SR_B6',
    'SR_B5']).rename('NDBI')
    #LST in kelvin, converting to celsius
    #ST_B10 → scale → Kelvin → Celsius
    #REF:
https://www.researchgate.net/post/How\_do\_i\_calculate\_Land\_Surface\_Temperature\_Landsat\_8\_level\_2\_image
    lst = image.select('ST_B10').subtract(273.15).rename('LST')
    return image.addBands([ndvi, ndbi, lst])

processed=scaled.map(add_indices)

<IPython.core.display.HTML object>

# Calculating Percentiles to get the high end, mid end and low end
percentiles = processed.select(['NDVI', 'NDBI',
'LST']).reduce(ee.Reducer.percentile([10, 50, 90]))

p10 = percentiles.select(['NDVI_p10', 'NDBI_p10',
'LST_p10']).clip(dhaka)
p50 = percentiles.select(['NDVI_p50', 'NDBI_p50',
'LST_p50']).clip(dhaka)
p90 = percentiles.select(['NDVI_p90', 'NDBI_p90',
'LST_p90']).clip(dhaka)

<IPython.core.display.HTML object>

# Check Ranges of All Percentiles
bands = ['NDVI_p10', 'NDVI_p50', 'NDVI_p90',
         'NDBI_p10', 'NDBI_p50', 'NDBI_p90',
         'LST_p10', 'LST_p50', 'LST_p90']

stats = percentiles.select(bands).reduceRegion(
    reducer=ee.Reducer.minMax(),
    geometry=dhaka,
    scale=30,
    maxPixels=1e13
)

# Pretty print
for band in bands:
    min_val = stats.get(band + '_min').getInfo()
    max_val = stats.get(band + '_max').getInfo()
    print(f"{band:12} : min = {min_val:.4f}    max = {max_val:.4f}")

<IPython.core.display.HTML object>

```

```

NDVI_p10      : min = -0.5717      max = 0.6207
NDVI_p50      : min = -0.2437      max = 0.8119
NDVI_p90      : min = -0.0028      max = 0.8796
NDBI_p10      : min = -0.7585      max = 0.0217
NDBI_p50      : min = -0.5671      max = 0.3450
NDBI_p90      : min = -0.3445      max = 0.4931
LST_p10       : min = -31.1149     max = 34.8187
LST_p50       : min = 22.7240      max = 52.1575
LST_p90       : min = 23.0334      max = 61.7356

```

*# This code is only to get good values for min max when plotting
commented as it takes a lot of time to run*

```

pLST = p50.select('LST_p50').reduceRegion(
    reducer=ee.Reducer.percentile([2, 98]),
    geometry=dhaka,
    scale=30,
    maxPixels=1e13
)

```

```
print(pLST.getInfo())
```

```

pNDVI = p50.select('NDVI_p50').reduceRegion(
    reducer=ee.Reducer.percentile([2, 98]),
    geometry=dhaka,
    scale=30,
    maxPixels=1e13
)

```

```
print(pNDVI.getInfo())
```

```

pNDBI = p50.select('NDBI_p50').reduceRegion(
    reducer=ee.Reducer.percentile([2, 98]),
    geometry=dhaka,
    scale=30,
    maxPixels=1e13
)

```

```
print(pNDBI.getInfo())
```

<IPython.core.display.HTML object>

```

{'LST_p50_p2': 24.685051872561246, 'LST_p50_p98': 33.55875602982125}
{'NDVI_p50_p2': -0.09782340283124245, 'NDVI_p50_p98':
0.6678189850383187}
{'NDBI_p50_p2': -0.35736852413310394, 'NDBI_p50_p98':
0.0644198442380445}

```

output from prev cell

```

# {'LST_p50_p2': 24.685051872561246, 'LST_p50_p98':
33.558756029821254}
# {'NDVI_p50_p2': -0.09782340283124245, 'NDVI_p50_p98':
0.6678189850383187}

```

```

# {'NDBI_p50_p2': -0.35736852413310394, 'NDBI_p50_p98':
0.06441984423804449}

vis_lst = {'min': 24.7, 'max': 33.6, 'palette': ['blue', 'cyan',
'yellow', 'red']}
vis_ndvi = {'min': -0.1, 'max': 0.67, 'palette': ['brown', 'yellow',
'green']}
vis_ndbi = {'min': -0.36, 'max': 0.06, 'palette': ['blue', 'white',
'red']}

Map.addLayer(p50.select('LST_p50'), vis_lst, 'LST p50')
Map.addLayer(p50.select('NDVI_p50'), vis_ndvi, 'NDVI p50')
Map.addLayer(p50.select('NDBI_p50'), vis_ndbi, 'NDBI p50')

Map

<IPython.core.display.HTML object>

{"model_id": "4ee8608e835f4d4f8444d1eb89b2d0a0", "version_major": 2, "vers
ion_minor": 0}

# Population

#using the new global2, 100m data from worldpop
def create_pop_image(year=2024):
    pop_ic =
ee.ImageCollection('projects/sat-io/open-datasets/WORLDPPOP/pop')
    pop_img = (pop_ic
                .filter(ee.Filter.stringContains('system:index',
f'_POP_{year}_'))
                .select([0], ['population'])
                .mosaic()
                .clip(dhaka))
    return pop_img

# Load population
pop_image = create_pop_image(2024)
print("Population layer ready for Dhaka district.")

# Total population check
total_pop = pop_image.reduceRegion(
    reducer=ee.Reducer.sum(),
    geometry=dhaka,
    scale=100,
    maxPixels=1e13
).getInfo()

print(f"Total Population in Dhaka District (2024):
{total_pop['population']:, .0f}")

<IPython.core.display.HTML object>

```

Population layer ready for Dhaka district.
Total Population in Dhaka District (2024): 13,387,746

```
def create_pop_image(year):
    pop_ic =
ee.ImageCollection("projects/sat-io/open-datasets/WORLDPPOP/pop")
    pop_img = (pop_ic
                .filter(ee.Filter.stringContains('system:index',
f'_POP_{year}_'))
                .select([0], ['population'])
                .mosaic()
                .clip(dhaka))
    return pop_img

years = [2023, 2024, 2025]
for y in years:
    pop = create_pop_image(y)
    total = pop.reduceRegion(ee.Reducer.sum(), dhaka,
100).get('population').getInfo()
    print(f"Year {y}: {total:,.0f} people")

<IPython.core.display.HTML object>

Year 2023: 13,105,147 people
Year 2024: 13,387,746 people
Year 2025: 13,678,728 people

#extract median for all indices
p50_clean = (processed.select(['NDVI', 'NDBI', 'LST'])
              .reduce(ee.Reducer.percentile([50]))
              .select(['NDVI_p50', 'NDBI_p50', 'LST_p50'])
              .clip(dhaka))

# Population-weighted calculation
ndvi = p50_clean.select('NDVI_p50')
ndbi = p50_clean.select('NDBI_p50')
lst = p50_clean.select('LST_p50')

#keep only pixels where population > 0
pop_mask = pop_image.gt(0)

# Create weighted bands (population x value) for each pixel
ndvi_w =
ndvi.updateMask(pop_mask).multiply(pop_image).rename('NDVI_w')
ndbi_w =
ndbi.updateMask(pop_mask).multiply(pop_image).rename('NDBI_w')
lst_w = lst.updateMask(pop_mask).multiply(pop_image).rename('LST_w')

combined = ndvi_w.addBands([ndbi_w, lst_w,
pop_image.rename('population')])
```

```

stats = combined.reduceRegion(
    reducer=ee.Reducer.sum(),
    geometry=dhaka,
    scale=100,
    maxPixels=1e13,
    tileScale=4
)

total_pop_num = ee.Number(stats.get('population'))

weighted_results = {
    'NDVI_pop_weighted_p50':
ee.Number(stats.get('NDVI_w')).divide(total_pop_num),
    'NDBI_pop_weighted_p50':
ee.Number(stats.get('NDBI_w')).divide(total_pop_num),
    'LST_pop_weighted_p50':
ee.Number(stats.get('LST_w')).divide(total_pop_num),
    'total_population': total_pop_num,
    'simple_NDVI_mean': ndvi.reduceRegion(ee.Reducer.mean(), dhaka,
30).get('NDVI_p50'),
    'simple_NDBI_mean': ndbi.reduceRegion(ee.Reducer.mean(), dhaka,
30).get('NDBI_p50'),
    'simple_LST_mean': lst.reduceRegion(ee.Reducer.mean(), dhaka,
30).get('LST_p50')
}

```

<IPython.core.display.HTML object>

```

print("\n" + "="*60)
print("POPULATION-WEIGHTED vs SIMPLE MEAN (p50)")
print("="*60)
results = weighted_results
for key, value in results.items():
    if isinstance(value, ee.ComputedObject):
        val = value.getInfo()
        if isinstance(val, float):
            print(f"{key:25}: {val:.4f}")
        else:
            print(f"{key:25}: {val}")
    else:
        print(f"{key:25}: {value:.4f}")

```

OUTPUT

```

# =====
# POPULATION-WEIGHTED vs SIMPLE MEAN (p50)
# =====
# NDVI_pop_weighted_p50      : 0.3976
# NDBI_pop_weighted_p50      : -0.1034
# LST_pop_weighted_p50       : 30.3314

```



```
# total_population      : 13387746.4420
# simple_NDVI_mean      : 0.4465
# simple_NDBI_mean      : -0.1790
# simple_LST_mean       : 29.2500
```

<IPython.core.display.HTML object>

```
=====
POPULATION-WEIGHTED vs SIMPLE MEAN (p50)
=====
```

```
NDVI_pop_weighted_p50    : 0.3976
NDBI_pop_weighted_p50    : -0.1034
LST_pop_weighted_p50     : 30.3314
total_population         : 13387746.4420
simple_NDVI_mean          : 0.4465
simple_NDBI_mean          : -0.1790
simple_LST_mean           : 29.2500
```

```
# import ee
```

```
# # Make sure Earth Engine is initialized
# ee.Initialize(project='qgis-proj') # or whatever project ID you're
using
```

```
# # =====
```

```
# # Export Settings
```

```
# # =====
```

```
# export_region = dhaka # your Dhaka geometry (already defined)
```

```
# export_opts = {
#     'region': export_region,
#     'crs': 'EPSG:32645',      # ← Corrected: Dhaka is in UTM 45N,
not 46N
#     'maxPixels': 1e13,      # Increased (safer for large area)
#     'fileFormat': 'GeoTIFF'
# }
```

```
# folder_name = 'DHAKA_FINAL_RASTERS'
```

```
# # =====
```

```
# # 1. Export WorldPop (100m)
```

```
# # =====
```

```
# pop_image = create_pop_image(2024) # reuse your existing function
```

```
# task_pop = ee.batch.Export.image.toDrive(
#     image=pop_image,
#     description='Dhaka_WorldPop2024',
#     folder=folder_name,
#     fileNamePrefix='Dhaka_WorldPop2024',
#     scale=100,
```

```

#     region=export_region,
#     crs=export_opts['crs'],
#     maxPixels=export_opts['maxPixels']
# )
# task_pop.start()
# print("□ WorldPop export started...")

# # =====
# # 2. Export p50 Rasters (Main analysis)
# # =====
# p50 = percentiles.select(['NDVI_p50', 'NDBI_p50',
# 'LST_p50']).clip(dhaka)

# task_ndvi = ee.batch.Export.image.toDrive(
#     image=p50.select('NDVI_p50'),
#     description='Dhaka_NDVI_p50',
#     folder=folder_name,
#     fileNamePrefix='Dhaka_NDVI_p50',
#     scale=30,
#     region=export_region,
#     crs=export_opts['crs'],
#     maxPixels=export_opts['maxPixels']
# )
# task_ndvi.start()

# task_ndbi = ee.batch.Export.image.toDrive(
#     image=p50.select('NDBI_p50'),
#     description='Dhaka_NDBI_p50',
#     folder=folder_name,
#     fileNamePrefix='Dhaka_NDBI_p50',
#     scale=30,
#     region=export_region,
#     crs=export_opts['crs'],
#     maxPixels=export_opts['maxPixels']
# )
# task_ndbi.start()

# task_lst = ee.batch.Export.image.toDrive(
#     image=p50.select('LST_p50'),
#     description='Dhaka_LST_p50',
#     folder=folder_name,
#     fileNamePrefix='Dhaka_LST_p50',
#     scale=30,
#     region=export_region,
#     crs=export_opts['crs'],
#     maxPixels=export_opts['maxPixels']
# )
# task_lst.start()

```

```

# # =====
# # 3. (Optional but Recommended) Export p90 for extremes
# # =====
# task_lst_p90 = ee.batch.Export.image.toDrive(
#     image=p90.select('LST_p90'),
#     description='Dhaka_LST_p90',
#     folder=folder_name,
#     fileNamePrefix='Dhaka_LST_p90',
#     scale=30,
#     region=export_region,
#     crs=export_opts['crs'],
#     maxPixels=export_opts['maxPixels']
# )
# task_lst_p90.start()

# To check the status of tasks written in previous block
# for t in ee.batch.Task.list():
#     print(t.status()['state'], t.config.get('description'))

```